c:\hpc-23-feb-2026\.\38\file_creator.c

```
#include<stdio.h>
#include<string.h>
int main()
{
char buffer[1024];
FILE *f;
int x;
memset(buffer,'A',sizeof(buffer));
f=fopen("test.data","wb");
for(x=1;x<5*1024*1024;++x)
{
fwrite(buffer,sizeof(buffer),1,f);
}
fclose(f);
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\38\serial_reader.cpp

```
#include<iostream>
#include<stdio.h>
#include<chrono>
using namespace std;
int main()
{
char *buffer;
buffer=new char[4096];
FILE *f;
int read_count;
char c;
int i;
auto start=chrono::high_resolution_clock::now();
f=fopen("test.data","rb");
while(1)
{
read_count=fread(buffer,1,sizeof(buffer),f);
if(read_count==0) break;
for(i=0;i<read_count;++i) c=buffer[i];
}
fclose(f);
auto end=chrono::high_resolution_clock::now();
auto d=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<d.count()<<" milliseconds"<<endl;
return 0;
}
```

Thinking Machines Classroom Codes

c:\hpc-23-feb-2026\.\38\eg215.cpp

```
#include<iostream>
#include<unistd.h>
using namespace std;
int main()
{
size_t page_size=sysconf(_SC_PAGESIZE);
cout<<page_size<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\38\read_mmap.cpp

```
#include<iostream>
#include<sys/stat.h>
#include<unistd.h>
#include<fcntl.h>
#include<sys/mman.h> // for functions related to memory management, right now (mmap)
#include<chrono>
using namespace std;
using namespace std::chrono;
int main()
{
auto start=high_resolution_clock::now();
struct stat s;
stat("test.data",&s);
size_t file_size=s.st_size;
size_t page_size=sysconf(_SC_PAGESIZE);
cout<<"File size : "<<file_size<<endl;
cout<<"System page size : "<<page_size<<endl;
int file_descriptor;
file_descriptor=open("test.data",O_RDONLY);
off_t offset;
size_t chunk_size=page_size;
size_t how_many_bytes_to_map;
how_many_bytes_to_map=chunk_size;
offset=0;
char c;
int i;
char *ptr;
while(offset<file_size)
{
if((offset+(off_t)how_many_bytes_to_map) >file_size) how_many_bytes_to_map=file_size-offset;
ptr=(char *)mmap(NULL,how_many_bytes_to_map,PROT_READ,MAP_PRIVATE,file_descriptor,offset);
if(ptr==MAP_FAILED)
{
cout<<"Unable to map"<<endl;
return 1;
}
for(i=0;i<how_many_bytes_to_map;++i) c=ptr[i];
munmap(ptr,how_many_bytes_to_map);
offset+=how_many_bytes_to_map;
}
close(file_descriptor);
auto end=high_resolution_clock::now();
auto d=duration_cast<milliseconds>(end-start);
cout<<d.count()<<" milliseconds"<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\38\read_mmap2.cpp

```
#include<iostream>
#include<sys/stat.h>
#include<unistd.h>
#include<fcntl.h>
#include<sys/mman.h> // for functions related to memory management, right now (mmap)
#include<chrono>
using namespace std;
using namespace std::chrono;
int main()
{
auto start=high_resolution_clock::now();
struct stat s;
stat("test.data",&s);
size_t file_size=s.st_size;
cout<<"File size : "<<file_size<<endl;
int file_descriptor;
file_descriptor=open("test.data",O_RDONLY);
off_t offset;
size_t how_many_bytes_to_map;
how_many_bytes_to_map=file_size;
offset=0;
char c;
int i;
char *ptr;
offset=0;
ptr=(char *)mmap(NULL,how_many_bytes_to_map,PROT_READ,MAP_PRIVATE,file_descriptor,offset);
if(ptr==MAP_FAILED)
{
cout<<"Unable to map"<<endl;
return 1;
}
for(i=0;i<how_many_bytes_to_map;++i) c=ptr[i];
munmap(ptr,how_many_bytes_to_map);
close(file_descriptor);
auto end=high_resolution_clock::now();
auto d=duration_cast<milliseconds>(end-start);
cout<<d.count()<<" milliseconds"<<endl;
return 0;
}
```